HYPERTUBE

Extreme Programming Cards

Stack: Rust (Axum) · React (Vite + TypeScript) · PostgreSQL · Redis · Docker

Objetivo: mínimo para passar — 100% test coverage em cada card

CARD 01 — Infraestrutura Docker

Épico	Infraestrutura
Pontos	3
Prioridade	P0 — Deve ser feito primeiro
Dependências	Nenhuma

User Story

- Como desenvolvedor, quero um ambiente Docker Compose totalmente reproduzível que suba todos os serviços com um único comando, para que o projeto funcione igual em qualquer máquina durante a defesa.

Acceptance Criteria

- docker compose up --build sobe todos os containers sem erro
- Frontend React acessível em http://localhost:5173
- Backend Rust acessível em http://localhost:8000
- PostgreSQL acessível internamente na porta 5432
- Redis acessível internamente na porta 6379
- aria2c rodando em modo daemon RPC na porta 6800
- Volume compartilhado /downloads acessível por backend e aria2
- Arquivo .env existe, está no .gitignore e contém todas as variáveis sensíveis
- Arquivo .env.example existe com todas as chaves (sem valores reais) como documentação

Tests (100% Coverage)

- √ TEST: docker compose up --build termina com exit code 0
- √ TEST: curl http://localhost:8000/health retorna 200 OK
- √ TEST: curl http://localhost:5173 retorna HTML
- √ TEST: verificar que .env está listado no .gitignore
- √ TEST: verificar que .env.example existe e contém todas as chaves
- √ TEST: git log --all -- .env não retorna nenhum commit (nunca foi commitado)
- √ TEST: aria2c RPC responde em http://localhost:6800/jsonrpc

Eliminatory / Critical Rules

⚠ NUNCA commitar .env — resultado automático 0 no projeto
⚠ Qualquer credencial pública = cheat flag na avaliação

CARD 02 — Schema do Banco de Dados

Épico	Infraestrutura
Pontos	2
Prioridade	P0
Dependências	Card 01

User Story

- Como desenvolvedor, quero o schema SQL inicial com todas as tabelas necessárias para o projeto, aplicado automaticamente via migrations ao subir o Docker.

Acceptance Criteria

- Migration roda automaticamente ao iniciar o container do backend
- Tabela users: id (uuid), email, username, first_name, last_name, password_hash, profile_picture_url, preferred_language, created_at, updated_at
- Tabela oauth_accounts: id, user_id (fk), provider (42|google), provider_user_id, created_at
- Tabela password_reset_tokens: id, user_id (fk), token_hash, expires_at, used_at
- Tabela movies: id (uuid), title, year, imdb_id, imdb_rating, genre, length_minutes, cover_url, torrent_magnet, torrent_hash, file_path, downloaded_at, last_watched_at, created_at
- Tabela watched_movies: id, user_id (fk), movie_id (fk), watched_at
- Tabela comments: id (uuid), user_id (fk), movie_id (fk), content, created_at, updated_at
- Tabela oauth2_clients: id, client_id, client_secret_hash, name, created_at
- Tabela oauth2_tokens: id, client_id (fk), user_id (fk, nullable), token_hash, expires_at, created_at
- Todos os campos sensíveis nunca armazenam texto puro

Tests (100% Coverage)

√ TEST: sqlx migrate run termina sem erro
√ TEST: todas as 8 tabelas existem após migration
√ TEST: coluna password_hash NÃO aceita insert de texto plain (constraint check via aplicação)
√ TEST: foreign keys estão configuradas — insert de movie_id inválido em comments retorna erro
√ TEST: rollback da migration funciona (migrate revert)
√ TEST: duas execuções de migrate run são idempotentes (não falha na segunda)

CARD 03 — Registro de Usuário

Épico	Autenticação
Pontos	3
Prioridade	P0
Dependências	Card 02

╗ User Story

- Como visitante, quero me registrar fornecendo email, username, primeiro nome, último nome, foto de perfil e senha, para ter acesso à plataforma.

≠ Acceptance Criteria

- POST /api/auth/register aceita: email, username, first_name, last_name, password, profile_picture (multipart ou URL)
- Senha é armazenada como hash bcrypt com custo mínimo 12 — NUNCA em texto puro
- Validação: email deve ser formato válido (regex RFC 5322 básico)
- Validação: username entre 3 e 20 caracteres, apenas letras/números/underscores
- Validação: first_name e last_name entre 1 e 50 caracteres
- Validação: senha mínimo 8 caracteres, deve conter letra maiúscula, minúscula e número
- Validação: email duplicado retorna 409 Conflict com mensagem clara
- Validação: username duplicado retorna 409 Conflict com mensagem clara
- Upload de foto: aceitar apenas image/jpeg, image/png, image/webp, tamanho máximo 5MB
- Upload de foto: arquivo é renomeado para UUID gerado pelo servidor (nunca usar nome original)
- Resposta de sucesso: 201 Created com o objeto user (sem password_hash)
- Frontend: formulário exibe mensagem de erro por campo com falha de validação
- Frontend: após registro bem-sucedido, redirecionar para login

---

к Tests (100% Coverage)

- √ TEST: POST /api/auth/register com dados válidos retorna 201 e user sem password_hash
- √ TEST: verificar no banco que password_hash != payload.password
- √ TEST: POST com email já existente retorna 409
- √ TEST: POST com username já existente retorna 409
- √ TEST: POST com email inválido retorna 422 com campo 'email' no erro
- √ TEST: POST com senha sem maiúscula retorna 422
- √ TEST: POST com senha com menos de 8 chars retorna 422
- √ TEST: POST com username com espaço retorna 422
- √ TEST: upload de arquivo .exe retorna 422 (tipo não permitido)
- √ TEST: upload de arquivo PNG com 6MB retorna 413
- √ TEST: arquivo salvo no servidor tem nome UUID, não o nome original
- √ TEST: campo password_hash não aparece em nenhuma resposta da API

⚠ Eliminatory / Critical Rules

- ⚠ Senha em plain text no banco = ZERO imediato na avaliação
- ⚠ Upload sem validação de MIME real = falha de segurança eliminatória
- ⚠ SQL injection: usar sqlx query! macros, nunca concatenar strings SQL

CARD 04 — Login com Credenciais + Logout

Épico	Autenticação
Pontos	2
Prioridade	P0
Dependências	Card 03

╗ User Story

- Como usuário registrado, quero fazer login com meu username/email e senha, receber um token de sessão, e poder fazer logout de qualquer página.

≠ Acceptance Criteria

- POST /api/auth/login aceita username (ou email) + password
- Verifica bcrypt hash — nunca compara texto puro
- Retorna JWT com payload: user_id, username, exp (24h)
- JWT_SECRET armazenado apenas no .env
- Frontend armazena o JWT em memória (estado React/Zustand) — NÃO em localStorage
- Todas as rotas protegidas verificam o header Authorization: Bearer <token>
- Token inválido ou expirado retorna 401 Unauthorized
- Logout no frontend limpa o estado do token e redireciona para /login
- Botão de logout visível em toda página quando logado (header)
- POST /api/auth/login com credenciais erradas retorna 401 (sem indicar qual campo é errado)

ϰ Tests (100% Coverage)

- √ TEST: POST /api/auth/login com credenciais válidas retorna 200 e token JWT
- √ TEST: decodificar JWT e verificar que contém user_id e exp
- √ TEST: POST /api/auth/login com senha errada retorna 401
- √ TEST: POST /api/auth/login com username inexistente retorna 401
- √ TEST: GET /api/users (rota protegida) sem token retorna 401
- √ TEST: GET /api/users com token expirado (exp no passado) retorna 401
- √ TEST: GET /api/users com token válido retorna 200
- √ TEST: após logout (frontend), nova tentativa de acessar rota protegida redireciona para login

CARD 05 — OAuth2 via 42 School

Épico	Autenticação
Pontos	3
Prioridade	P0
Dependências	Card 04

ɳ User Story

- Como usuário da 42, quero me registrar e logar usando minha conta 42 via OAuth2, sem precisar criar uma senha manual.

≠ Acceptance Criteria

- GET /api/auth/oauth/42 redireciona o usuário para a página de autorização da 42
- Callback configurado: http://localhost:3000/api/auth/oauth/42/callback
- Após autorização da 42, o callback recebe o code e faz exchange pelo access_token
- Com o access_token, busca os dados do usuário na API da 42 (email, login, first_name, last_name, image.link)
- Se o usuário não existe no banco: cria automaticamente com os dados da 42 (sem senha — password_hash NULL)
- Se o usuário já existe (mesmo email): vincula a conta OAuth ao usuário existente na tabela oauth_accounts
- Cria JWT e redireciona para o frontend com o token
- CLIENT_ID e CLIENT_SECRET da 42 ficam APENAS no .env
- Variáveis necessárias no .env: OAUTH_42_CLIENT_ID, OAUTH_42_CLIENT_SECRET, OAUTH_42_REDIRECT_URI=http://localhost:3000/api/auth/oauth/42/callback

к Tests (100% Coverage)

- √ TEST: GET /api/auth/oauth/42 retorna 302 com Location para intra.42.fr/oauth/authorize
- √ TEST: callback com code válido cria usuário novo no banco (se não existia)
- √ TEST: callback com code válido para usuário existente cria registro em oauth_accounts
- √ TEST: callback retorna redirect para frontend com token JWT válido
- √ TEST: callback com code inválido/expirado retorna 400
- √ TEST: OAUTH_42_CLIENT_SECRET não aparece em nenhuma resposta HTTP
- √ TEST: usuário criado via 42 não tem password_hash no banco

⚠ Eliminatory / Critical Rules

- ⚠ CLIENT_SECRET da 42 no código ou git = cheat/zero automático

CARD 06 — OAuth2 via Google

Épico	Autenticação
Pontos	3
Prioridade	P0
Dependências	Card 04

ๅ User Story

- Como usuário, quero me registrar e logar usando minha conta Google via OAuth2, para ter uma segunda opção de autenticação social.

≠ Acceptance Criteria

- GET /api/auth/oauth/google redireciona para accounts.google.com/o/oauth2/v2/auth
- Callbacks configurados no Google Console:
- • http://localhost:8000/auth/google/callback
- • http://localhost:3000/api/auth/oauth/google/callback
- Origens JavaScript autorizadas no Google Console:
- • http://localhost:3000
- • http://localhost:8000
- • http://localhost:5173
- Após autorização, faz exchange pelo access_token e busca dados do usuário na API do Google
- Mesma lógica de criação/vinculação de conta do Card 05
- Variáveis necessárias no .env: OAUTH_GOOGLE_CLIENT_ID, OAUTH_GOOGLE_CLIENT_SECRET, OAUTH_GOOGLE_REDIRECT_URI
- Botão 'Login com Google' visível na página de login e registro

к Tests (100% Coverage)

- √ TEST: GET /api/auth/oauth/google retorna 302 com Location para accounts.google.com
- √ TEST: callback com code válido do Google cria usuário ou vincula conta
- √ TEST: callback retorna redirect para frontend com JWT válido
- √ TEST: OAUTH_GOOGLE_CLIENT_SECRET não vaza em nenhuma response
- √ TEST: usuário pode logar com Google após ter se registrado com credenciais (mesmo email)

⚠ Eliminatory / Critical Rules

⚠ Google CLIENT_SECRET no código ou git = zero automático
⚠ Configurar EXATAMENTE as URLs de callback e origens conforme captura de tela fornecida

CARD 07 — Reset de Senha por Email

Épico	Autenticação
Pontos	2
Prioridade	P1
Dependências	Card 04

ๆ User Story

- Como usuário que esqueceu sua senha, quero receber um email com link de reset, para poder criar uma nova senha sem precisar de suporte.

≠ Acceptance Criteria

- POST /api/auth/forgot-password aceita { email }
- Se email existe: gera token aleatório seguro (crypto::rand), salva hash SHA256 na tabela password_reset_tokens com expires_at = agora + 1h
- Envia email com link: http://localhost:3000/reset-password?token=<token_plain>
- Se email não existe: retorna 200 mesmo assim (não vazar informação sobre emails cadastrados)
- POST /api/auth/reset-password aceita { token, new_password }
- Valida: token existe, hash bate, não expirou, não foi usado
- Valida nova senha com as mesmas regras do registro
- Atualiza password_hash com novo bcrypt hash
- Marca token como used_at = now()
- Variáveis no .env: SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM
- Para desenvolvimento: usar Mailhog (container Docker) como SMTP local

к Tests (100% Coverage)

√ TEST: POST /api/auth/forgot-password com email válido retorna 200
√ TEST: email foi enviado para Mailhog (verificar via API do Mailhog)
√ TEST: link no email contém token válido
√ TEST: POST /api/auth/forgot-password com email inexistente também retorna 200
√ TEST: POST /api/auth/reset-password com token válido atualiza senha e retorna 200
√ TEST: login com nova senha funciona
√ TEST: login com senha antiga falha após reset
√ TEST: POST /api/auth/reset-password com token já usado retorna 400
√ TEST: POST /api/auth/reset-password com token expirado retorna 400
√ TEST: token salvo no banco é o hash SHA256, não o token plain

CARD 08 — Perfil de Usuário (Edição e Visualização)

Épico	Usuários

Pontos	2
Prioridade	P1
Dependências	Card 04

╗ User Story

- Como usuário logado, quero editar meus dados (email, username, foto, nome, idioma preferido) e visualizar o perfil de outros usuários, sem expor emails alheios.

≠ Acceptance Criteria

- GET /api/users/:id retorna: username, first_name, last_name, profile_picture_url, preferred_language — NÃO retorna email
- PATCH /api/users/:id (autenticado, só o próprio user) aceita: email, username, first_name, last_name, password, profile_picture_url, preferred_language
- Mesmas validações do registro se aplicam na edição
- Se novo email ou username já existe para outro user: retorna 409
- Idioma preferido: aceitar valores 'en' e 'pt' (mínimo dois idiomas)
- Frontend: página /profile com formulário de edição pré-preenchido com dados atuais
- Frontend: página /users/:id com visualização do perfil de outro usuário (sem email)
- Frontend: troca de idioma aplica react-i18next em toda a interface
- Upload de nova foto de perfil: mesmas validações do Card 03

к Tests (100% Coverage)

√ TEST: GET /api/users/:id retorna dados do usuário SEM campo email
√ TEST: PATCH /api/users/:id com token do próprio user atualiza dados com sucesso
√ TEST: PATCH /api/users/:id com token de outro user retorna 403
√ TEST: PATCH com email de outro user existente retorna 409
√ TEST: PATCH com preferred_language='pt' persiste no banco
√ TEST: interface exibe textos em PT após mudar idioma
√ TEST: interface exibe textos em EN (padrão) sem configuração
√ TEST: página /users/:id de outro usuário não exibe campo de email

CARD 09 — Busca de Vídeos (Archive.org + LegitTorrents)

Épico	Biblioteca
Pontos	5
Prioridade	P0
Dependências	Card 04

╗ User Story

- Como usuário logado, quero buscar vídeos por título e ver resultados de pelo menos duas fontes externas legais, para encontrar conteúdo para assistir.

≠ Acceptance Criteria

- GET /api/search?q=<query> consulta as duas fontes em paralelo (tokio::join!)

- Fonte 1: Archive.org — usar API: https://archive.org/advancedsearch.php?q=<query>&fl[]=identifier,title,year,subject&rows=20&output=json&mediatype=movies
- Fonte 2: LegitTorrents.info — scraping HTML com reqwest + scraper crate (busca por título)
- Resultado: array de objetos com { id, title, year, cover_url, magnet, source }
- Para cada resultado com título, buscar rating/metadados no OMDb API: http://www.omdbapi.com/?t=<title>&apikey=<key>
- Resultados do OMDb cacheados no Redis por 24h (key: omdb:<title>)
- Resultados de busca cacheados no Redis por 1h (key: search:<query>)
- Se nenhuma busca foi feita (página inicial), exibir populares do Archive.org: sort=-downloads
- Rota requer autenticação — sem token retorna 401
- Variável no .env: OMDB_API_KEY

κ Tests (100% Coverage)

√ TEST: GET /api/search?q=documentary retorna array com resultados de pelo menos 2 fontes
√ TEST: cada item do array tem title, year, cover_url, magnet
√ TEST: segunda chamada com mesmo query é servida do cache Redis
√ TEST: GET /api/search sem token retorna 401
√ TEST: GET /api/search?q= (vazio) retorna populares do Archive.org
√ TEST: OMDB_API_KEY não aparece em nenhuma response HTTP
√ TEST: requisições para Archive.org e LegitTorrents são feitas em paralelo (medir tempo < soma individual)

CARD 10 — Thumbnails, Paginação e Filtros

Épico	Biblioteca
Pontos	3
Prioridade	P1
Dependências	Card 09

η User Story

- Como usuário, quero ver os resultados como cards com informações visuais, poder filtrar/ordenar, e ter carregamento automático ao rolar a tela.

≠ Acceptance Criteria

- Cada thumbnail exibe: cover_url (imagem), title, year, imdb_rating, badge 'ASSISTIDO' se o usuário já viu
- Watched/unwatched: thumbnail de vídeo já assistido tem overlay escuro + badge 'Assistido'
- Paginação: API retorna cursor-based ou page+limit (page=1&limit=20)
- Infinite scroll: frontend usa IntersectionObserver em um div sentinela no final da lista
- Quando sentinela entra na viewport: fetch da próxima página e append nos itens existentes
- NÃO há botão 'Próxima Página' — o carregamento é automático
- Ordenação: dropdown com opções name (A-Z), year (mais recente), rating (maior)
- Filtros: por genre (se disponível do OMDb), por year range
- Ordenação e filtros são aplicados no frontend (os dados já foram carregados via busca)
- Layout responsivo: 4 colunas desktop, 2 tablets, 1 mobile

κ Tests (100% Coverage)

√ TEST: lista exibe thumbnails com título, ano e rating
√ TEST: vídeo marcado como assistido exibe badge/overlay diferente
√ TEST: ao rolar até o final da lista, nova página é carregada automaticamente
√ TEST: NÃO existe botão de próxima página no DOM

- √ TEST: ordenar por rating reordena os cards na tela
- √ TEST: filtrar por genre exibe apenas cards do gênero selecionado
- √ TEST: layout em 375px de largura exibe 1 coluna
- √ TEST: layout em 768px exibe 2 colunas
- √ TEST: layout em 1280px exibe 4 colunas

CARD 11 — Página de Detalhes do Vídeo

Épico	Vídeo
Pontos	3
Prioridade	P1
Dependências	Card 09

╗ User Story

- Como usuário, quero ver os detalhes completos de um vídeo (metadados, elenco, comentários) antes e durante a reprodução.

≠ Acceptance Criteria

- GET /api/movies/:id retorna: title, year, length, imdb_rating, genre, summary, director, cast[], cover_url, available_subtitles[], comments_count
- Dados de casting e summary buscados no OMDb (ou TMDb) e salvos no banco na primeira vez
- Página /movies/:id exibe: player de vídeo, cover, título, ano, duração, rating, gênero, resumo, diretor, elenco principal
- Seção de comentários abaixo do player: lista de comentários com username, data e texto
- Formulário para novo comentário (textarea + botão) — somente para usuários logados
- POST /api/movies/:id/comments aceita { content } — user_id vem do JWT
- Comentário salvo com created_at e retornado imediatamente na lista
- Rota de detalhes requer autenticação

╖ Tests (100% Coverage)

- √ TEST: GET /api/movies/:id retorna todos os campos obrigatórios
- √ TEST: GET /api/movies/:id sem token retorna 401
- √ TEST: página exibe summary, director e pelo menos 1 ator do elenco
- √ TEST: POST /api/movies/:id/comments com content válido cria comentário e retorna 201
- √ TEST: POST /api/movies/:id/comments sem token retorna 401
- √ TEST: POST /api/movies/:id/comments com content vazio retorna 422
- √ TEST: GET /api/movies/:id após comentar reflete o novo comentário
- √ TEST: comentário exibe username e data formatada

CARD 12 — Download via aria2 + Streaming Progressivo

Épico	Vídeo / Torrent
Pontos	8
Prioridade	P0 — Feature Central
Dependências	Cards 02, 11

User Story

- Como usuário, quero clicar em um vídeo e começar a assistir o mais rápido possível, enquanto o servidor baixa o arquivo via BitTorrent em background.

Acceptance Criteria

- POST /api/movies/:id/stream inicia o download se arquivo não existir
- Integração com aria2 RPC via JSON-RPC em http://aria2:6800/jsonrpc com token ARIA2_RPC_SECRET
- Payload para aria2: aria2.addUri com magnet link + opções: dir=/downloads, bt-prioritize-piece=head+tail, seed-time=0
- Backend salva o gid do aria2 no banco (movies.aria2_gid)
- GET /api/movies/:id/status retorna: { status: 'downloading'|'ready'|'not_started', progress: 0-100, file_path }
- Polling: frontend chama /status a cada 2 segundos enquanto status == 'downloading'
- Quando progress > 5% (ou primeiros 10MB baixados): status muda para 'ready' e streaming pode começar
- GET /api/movies/:id/stream serve o arquivo com suporte a Range requests (HTTP 206 Partial Content)
- Headers obrigatórios: Accept-Ranges: bytes, Content-Range: bytes start-end/total
- Se arquivo já existe no disco (baixado anteriormente): inicia streaming imediatamente, sem novo download
- Registro de watched: ao iniciar stream, inserir/atualizar watched_movies com watched_at = now()
- Variável no .env: ARIA2_RPC_SECRET

Tests (100% Coverage)

- √ TEST: POST /api/movies/:id/stream com magnet válido retorna 202 Accepted
- √ TEST: aria2c lista o download (aria2.tellActive ou aria2.tellWaiting)
- √ TEST: GET /api/movies/:id/status retorna status 'downloading' enquanto baixa
- √ TEST: GET /api/movies/:id/status retorna status 'ready' quando progress > 5%
- √ TEST: GET /api/movies/:id/stream com header Range: bytes=0-1023 retorna 206
- √ TEST: Content-Range header presente na resposta 206
- √ TEST: segundo clique em vídeo já baixado NÃO cria novo download no aria2
- √ TEST: watched_movies tem registro após iniciar streaming
- √ TEST: ARIA2_RPC_SECRET não aparece em nenhuma resposta HTTP
- √ TEST: o magnet de teste do marking sheet funciona: magnet:?xt=urn:btih:79816060ea56d56f2a2148cd45705511079f9bca&dn=TPB.AFK.2013.720p.h264-SimonKlose

⚠ Eliminatory / Critical Rules

- ⚠ webtorrent, peerflix ou pulsar no package.json/Cargo.toml = cheat automático
- ⚠ Download deve ser 100% server-side via aria2 — nunca no cliente
- ⚠ O avaliador VAI testar o magnet específico do marking sheet acima

CARD 13 — Conversão MKV → MP4 com FFmpeg

Épico	Vídeo / Conversão
Pontos	4
Prioridade	P0
Dependências	Card 12

User Story

- Como usuário, quero assistir vídeos em qualquer formato (especialmente MKV) diretamente no browser, mesmo que o formato original não seja compatível.

≠ Acceptance Criteria

- Após download completar, verificar extensão do arquivo: se .mkv (ou outro não-nativo), iniciar conversão
- Formatos nativos do browser (NÃO converter): .mp4, .webm
- Formatos a converter: .mkv → .mp4 (mínimo obrigatório pelo subject)
- FFmpeg executado como processo filho do Tokio: tokio::process::Command
- Comando FFmpeg: ffmpeg -i input.mkv -c:v copy -c:a aac -movflags faststart output.mp4
- Flag faststart move metadados para início do arquivo — permite streaming antes do fim da conversão
- Conversão acontece em background (tokio::spawn) — não bloqueia a resposta da API
- GET /api/movies/:id/status reflete status 'converting' durante a conversão
- Streaming só começa do arquivo convertido (.mp4) quando ele tem dados suficientes
- FFmpeg instalado no container do backend via Dockerfile (apt-get install -y ffmpeg)

к Tests (100% Coverage)

- √ TEST: download de arquivo .mkv dispara conversão automática para .mp4
- √ TEST: GET /api/movies/:id/status retorna status 'converting' durante FFmpeg
- √ TEST: arquivo .mp4 resultante é streamable pelo browser
- √ TEST: arquivo .mp4 já existente NÃO é reconvertido
- √ TEST: o magnet MKV do marking sheet (TPB.AFK.2013.720p.h264-SimonKlose) é streamável após conversão
- √ TEST: conversão não bloqueia outras requisições à API durante execução
- √ TEST: FFmpeg está disponível no container: docker exec backend ffmpeg -version

⚠ Eliminatory / Critical Rules

- ⚠ O avaliador vai testar ESPECIFICAMENTE o magnet .mkv do subject — é um caso eliminatório

CARD 14 — Limpeza Automática de Vídeos

Épico	Vídeo / Storage
Pontos	2
Prioridade	P2
Dependências	Card 12

ๅ User Story

- Como administrador do sistema, quero que vídeos não assistidos há mais de 1 mês sejam deletados automaticamente do servidor para economizar espaço.

≠ Acceptance Criteria

- Job Tokio que roda diariamente: tokio::time::interval(Duration::from_secs(86400))
- Query: SELECT id, file_path FROM movies WHERE last_watched_at < NOW() - INTERVAL '30 days' AND file_path IS NOT NULL
- Para cada resultado: std::fs::remove_file(file_path) + UPDATE movies SET file_path = NULL
- Log de cada arquivo deletado (tracing::info!)
- Ao tentar stream de vídeo sem file_path: reinicia download via aria2

ϰ Tests (100% Coverage)

- √ TEST: vídeo com last_watched_at = 31 dias atrás é deletado pelo job
- √ TEST: vídeo com last_watched_at = 29 dias atrás NÃO é deletado
- √ TEST: após deleção, file_path no banco é NULL
- √ TEST: ao tentar assistir vídeo deletado, novo download é iniciado
- √ TEST: job não falha se file_path já não existe no disco (erro silencioso)

CARD 15 — Legendas Automáticas

Épico	Vídeo / Legendas
Pontos	3
Prioridade	P2
Dependências	Card 12

ɳ User Story

- Como usuário, quero que legendas em inglês (e no meu idioma preferido, se disponível) sejam baixadas e selecionáveis no player de vídeo.

≠ Acceptance Criteria

- API de legendas: OpenSubtitles.com REST API (gratuita com registro)
- Buscar legendas por: imdb_id do vídeo (mais preciso) ou por título
- Sempre baixar legendas em inglês se disponíveis
- Se idioma preferido do usuário != inglês e legendas disponíveis: baixar também no idioma preferido
- Legendas salvas em /downloads/<movie_id>/<lang>.srt
- GET /api/movies/:id/subtitles retorna array: [{ lang: 'en', url: '/subtitles/<movie_id>/en.srt' }]
- Player de vídeo: elemento <track> com src apontando para as legendas
- Usuário pode selecionar legenda no controle nativo do <video>
- Variáveis no .env: OPENSUBTITLES_API_KEY, OPENSUBTITLES_USER_AGENT

ϰ Tests (100% Coverage)

- √ TEST: GET /api/movies/:id/subtitles retorna array com ao menos 1 legenda (para vídeo popular)
- √ TEST: arquivo .srt existe no servidor após fetch de legendas
- √ TEST: elemento <track> presente no HTML do player com src correto
- √ TEST: usuário com preferred_language='pt' tem legenda em PT disponível (se existir)
- √ TEST: OPENSUBTITLES_API_KEY não aparece em nenhuma resposta HTTP

CARD 16 — API RESTful com OAuth2 Server (Bonus)

Épico	API / Bonus
Pontos	5
Prioridade	P3 — Bonus (só avaliado se tudo mais perfeito)
Dependências	Cards 03, 08, 11

╗ User Story

- Como desenvolvedor externo, quero usar uma API RESTful autenticada via OAuth2 para acessar dados do Hypertube programaticamente.

≠ Acceptance Criteria

- POST /oauth/token aceita { client_id, client_secret, grant_type: 'client_credentials' } e retorna { access_token, token_type: 'Bearer', expires_in: 3600 }
- client_id e client_secret são pré-registrados na tabela oauth2_clients (seed no banco)
- GET /users → lista users com id e username (autenticado)
- GET /users/:id → username, email, profile_picture_url (autenticado)
- PATCH /users/:id → aceita username, email, password, profile_picture_url (autenticado, apenas o próprio user)
- GET /movies → lista de filmes da frontpage com id e name
- GET /movies/:id → name, id, imdb_rating, year, length, available_subtitles[], comments_count
- GET /comments → lista de comentários recentes: author_username, date, content, id
- GET /comments/:id → comment, author_username, comment_id, date_posted
- PATCH /comments/:id → aceita comment, username (autenticado, apenas autor)
- DELETE /comments/:id (autenticado, apenas autor)
- POST /comments → aceita comment, movie_id (autenticado)
- POST /movies/:movie_id/comments → aceita comment (autenticado)
- Qualquer rota não mapeada retorna 404 ou 405 — NUNCA 500

к Tests (100% Coverage)

- √ TEST: POST /oauth/token com client_id e client_secret válidos retorna access_token
- √ TEST: GET /users sem Bearer token retorna 401
- √ TEST: GET /users com Bearer token válido retorna array de users
- √ TEST: GET /movies retorna lista com id e name
- √ TEST: GET /movies/:id retorna todos os campos obrigatórios
- √ TEST: POST /comments cria comentário e retorna 201
- √ TEST: DELETE /comments/:id de outro usuário retorna 403
- √ TEST: GET /rota-inexistente retorna 404
- √ TEST: PUT /users retorna 405 Method Not Allowed
- √ TEST: access_token expirado (manipular exp) retorna 401

CARD 17 — Segurança e Hardening Geral

Épico	Segurança — Eliminatório
Pontos	Obrigatório — sem pontos, mas sem isso = zero
Prioridade	P0 — Verificado em TODA defesa
Dependências	Todos os cards

╗ User Story

- Como avaliador, vou verificar que o projeto não tem vulnerabilidades básicas que resultariam em nota zero imediata.

≠ Acceptance Criteria

- Senhas: NUNCA armazenar em plain text — sempre bcrypt com custo >= 12
- SQL Injection: usar EXCLUSIVAMENTE sqlx query! macros com bind parameters — zero string concatenation em queries

- XSS: frontend escapa todo conteúdo dinâmico — React já faz isso por padrão com JSX, mas verificar dangerouslySetInnerHTML (proibido)
- Upload: validar MIME type real lendo os bytes do arquivo (não só extensão), limitar tipos e tamanho
- CORS: configurar tower-http::cors para aceitar apenas http://localhost:5173 no desenvolvimento
- Secrets: .env no .gitignore desde o commit inicial, .env.example documentado
- JWT: secret forte (mínimo 32 chars aleatórios), nunca hardcoded no código
- Headers de segurança: X-Content-Type-Options: nosniff, X-Frame-Options: DENY (tower-http)
- Rate limiting: proteção básica no login (máx 5 tentativas por IP por minuto) — usar Redis counter
- Console limpo: zero warnings, errors ou notices no browser console durante toda a defesa
- Zero erros 5xx em operações normais da aplicação

κ Tests (100% Coverage)

√ TEST: inserir <script>alert(1)</script> num comentário e verificar que NÃO executa
√ TEST: tentar SQL injection em login: username = ' OR '1'='1 — deve retornar 401, não 200
√ TEST: upload de arquivo .php renomeado para .jpg deve ser rejeitado (MIME check)
√ TEST: acessar variáveis de ambiente via rota não deve ser possível
√ TEST: 6 tentativas de login com senha errada no mesmo IP deve bloquear a 6ª
√ TEST: browser console deve estar limpo em todas as páginas
√ TEST: todas as rotas protegidas retornam 401 sem token (não 500)
√ TEST: PATCH /users/:id com token de outro user retorna 403 (não 500)
√ TEST: git log --all -- .env não mostra nenhum commit

⚠ Eliminatory / Critical Rules

⚠ Plain text no banco = ZERO
⚠ SQL injection possível = ZERO
⚠ Credencial no git = CHEAT flag
⚠ Qualquer erro 5xx durante a defesa = CRASH flag
⚠ Warning no console do browser = CRASH flag

CARD 18 — Layout, Responsividade e UX

Épico	Frontend / UI
Pontos	2
Prioridade	P1
Dependências	Todos os cards de frontend

╗ User Story

- Como avaliador, quero que o site tenha uma estrutura mínima aceitável com header, main e footer, seja responsivo em mobile e funcione no Firefox e Chrome mais recentes.

≠ Acceptance Criteria

- Header presente em TODAS as páginas: logo/nome do site, link para biblioteca, botão de logout (se logado)
- Footer presente em TODAS as páginas: texto básico (ex: '© 2024 Hypertube')
- Main section: conteúdo principal em área central
- Responsivo: layout funcional em 375px (iPhone SE), 768px (iPad), 1280px (desktop)
- Compatível com Firefox >= 41 e Chrome >= 46 (usar apenas CSS e JS compatíveis)
- Sem uso de resources externos não carregados (zero broken images, zero 404 de assets)
- i18n: interface disponível em inglês (padrão) e pelo menos outro idioma

- Formulários: todos com labels, placeholders e mensagens de erro acessíveis

---

ϰ Tests (100% Coverage)

√ TEST: header está presente no DOM de todas as páginas (/, /login, /register, /library, /movies/:id)
√ TEST: footer está presente no DOM de todas as páginas
√ TEST: em 375px, nenhum elemento tem overflow horizontal
√ TEST: em 375px, textos são legíveis (font-size >= 14px)
√ TEST: abrir no Firefox em modo de compatibilidade — sem erros no console
√ TEST: abrir no Chrome — sem erros no console
√ TEST: todos os assets (imagens, fontes, CSS) retornam 200
√ TEST: trocar idioma no perfil muda os textos da interface